

Interim Individual Report

Team 2: Low Cost Perfusion Pump, *PulsePump*

Part IIA: GM1 *Multidisciplinary Design*

Tahmid Azam¹

with teammates Luca Buxton, India Henry, Lonnie McKiernan, and Lucy Payne
mentored by Dr. Peter Long and Prof. Athina E. Markaki

¹Department of Engineering, University of Cambridge

20 March 2026

We chose a waterfall software development model with the capacity for more interactive development during integration with control. The software requirements were developed further from the short team report, due to recognising the need to have effective software designed for use by the rest of the team to connect and test peripherals (stepper motors, servos, pressure sensors), and view and debug signals. Two use cases developed, for which solution-neutral problem statements involve:

1. a means to configure a collection of actuators and sensors, observe their behaviour as it happens, and preserve a record of it for later review; and
2. a means to specify a target pressure waveform for an artificial pump, expressed in a compact, human-readable form that can be shared between researchers and reproduced from published descriptions.

We better understood the lifecycle of the processes associated with running a session, a user story:

1. The researcher configures a particular target pressure waveform for a session of specified duration (e.g. 90 min).
2. The researcher connects the Raspberry Pico via USB, sends this to the Raspberry Pico, and begins the trial, with the ability to monitor the session live (viewing live plots of the target and actual pressure).
3. The researcher unplugs the Raspberry Pico from their device.
4. The researcher may return during the session, plugging in the Raspberry Pico, in order to check how the session is going.
5. The researcher may return long after the session has finished, and can download the full logs of all signals collected by the Pico during the run.
6. The researcher can provide details on the configuration of the pump in their literature as human-readable, serialised files.

We defined a `Programme` representing a complete description of a target pressure profile for a duration of time, composed of `Blocks`, which can be defined and generated by a source. Each `Block` specifies the number of cycles for which it is repeated. Sources currently include: a `FunctionGenerator`, which supports sinusoidal, pulse, square, triangular, constant, and sawtooth waves; and `openBF`, an open-source blood flow simulator. The `Programme` also defines the pressure unit and the sample rates. This implementation utilises run-length encoding (RLE) to dramatically reduce the file sizes required to be transferred over to the Raspberry Pico: instead of storing an array of all of the samples for the whole duration, we define a block and how many times it repeats, storing only one cycle's samples as a base-64 float32 byte string. The `Programme` is serialised to the YAML file format so definitions of each `Block` can be read by humans without the user interface.

Rather than implement the modelling pipeline from scratch, we have since opted to integrate `openBF`, an open-source 1D Navier-Stokes blood flow solver, allowing the solo development effort to be redirected toward software requirements analysis, user interface design, and the ergonomics of the tool. The motivation for this change of heart was the time constraints surrounding the project. The abstraction of `Blocks` being driven by a source with a set of configurable parameters also allows individuals to extend the functionality of the project by adding new sources (e.g. other blood flow simulation tools).

The design involves a 3-panel layout: a sidebar, a preview, and an inspector. On the left is a sidebar listing all of the blocks in the file. Clicking on a block moves the preview panel's focus to one cycle of the selected block. The preview renders the target pressure profile for the duration of the session, the keyboard shortcut `Mod+0` rescales to fit the whole session in the preview. The inspector panel is where the `Programme` and selected blocks are configured. In the menu bar, standard file operations are included: `New`, `Open`, `Save...`, `Save As...`; along with their standard operating system keyboard shortcuts. You can switch the entire user interface to just view the source YAML file under the View Menu.

We better understood the user stories behind members of the team running tests of peripheral devices connected to the Pico:

1. The engineer connects the Raspberry Pico their device and can verify that the firmware has been flashed and a bidirectional communication channel is opened.
2. The engineer connects peripheral devices (e.g. pressure sensors, stepper motors, servos) to the Raspberry Pico.
3. The engineer configures these devices (e.g. datasheet constants, pin assignment) in the software, and is warned if incompatible pins are selected (e.g. UART-capable, ADC-capable).

4. The engineer can view all data readouts from their peripheral devices, and can configure live plots in the user interface to visualise these signals.
5. The engineer starts a recording session, carries out tests, and ends it, saving all information related to the run in a folder on disk for later reference.

We designed a panel-based layout so each team member can easily customise the interface to better suit them. The panels include:

- A central signals panel (this is not removable), which lists each device and the signals it emits. For example, for a pressure sensor this is the raw counts of the ADC (a 16-bit binary value), along with this value transformed as a voltage, and a pressure according to the pressure sensor's datasheet.
- A connection panel which manages selecting the port to communicate to the Raspberry Pico on, and confirms successful connection.
- A run panel that allows for the starting and stopping of runs, and easy opening of the run folder in the operating system file explorer.
- A plot panel that allows for the dragging and dropping of signals from the signals panel for live plotting. Plotting panels can be added and removed such that there are more than 1 on the screen. They can be opened as separate windows too.
- A peripheral panel for each type of peripheral (e.g. stepper motor, pressure sensor, servo) that allows for the configuration of each peripheral (datasheet values, pin assignments).

A bottom status bar that reports any errors and confirms updates to the Pico's internal state.

We chose a technology stack by evaluating against project requirements: first enumerating and weighting selection criteria (Table 1), researching candidates (Table 2), and constructing a Pugh matrix (Table 3). From the Pugh matrix, we chose Qt/PySide6/PyQtGraph. It has a mature, panel-based layout engine, allowing for customisable and flexible docking of panels in various positions, and the opening of panels as separate windows as an extra benefit.

Table 1: Weighted selection criteria for GUI technology stack

criteria	description	weight
Cross-platform support	Runs on Windows, macOS, and Linux	5
Single codebase	Avoids fragmentation across platforms	5
UI framework maturity	Breadth and stability of UI components	4
Scientific (plotting) capability	Quality and performance of visualisation tools and scientific computing functions	5
Developer familiarity	Existing team experience with stack	4
Python/MicroPython compatibility	Enables code reuse with Pico firmware (DRY principle)	5
Development velocity	Iteration speed, tooling, and DX	4
Packaging/deployment	Ease of distribution and environment management	3

Table 2: Candidate GUI technology stacks: strengths and weaknesses

stack	strengths	weaknesses
Qt/PySide6/PyQtGraph	Mature desktop framework; excellent scientific ecosystem integration; high-performance real-time plotting; rapid iteration	Python packaging complexity; higher memory usage
Electron/React/Plotly	Modern web ecosystem; fast UI development; strong cross-platform distribution	High runtime overhead; weak scientific integration; no MicroPython parity
Qt/C++	Maximum performance; mature tooling; deep hardware integration	Slow iteration; high engineering complexity
Avalonia UI/C#	Modern cross-platform .NET UX; good desktop tooling	Smaller ecosystem; weaker scientific support
GTK/Python	Strong Linux integration; Python scientific ecosystem access	Less polished Windows/macOS UX
wxPython	Lightweight; native look-and-feel	Smaller ecosystem; dated tooling
Web application only	Highly portable; simple deployment; modern UI patterns	Limited hardware access; offline limitations

Table 3: Pugh matrix weighted scores by stack

stack	weighted score
Qt/PySide6/PyQtGraph	166
Web application only	143
Electron/React/Plotly	135
GTK/Python	131
wxPython	130
Qt/C++	124
Avalonia/C#	108

Development tooling is critical for LLM-based agentic coding platforms to have feedback on linting (styling) errors, compilation errors (N.B., in interpreted Python this more concerns itself with type errors), and allows LLMs to rely on external tools for code formatting. Running linting, formatting, and typechecking in continuous integration/continuous deployment pipelines prohibits the pushing of unsafe code that does not conform to best Python practices. In addition, as development is collaborative and takes place on multiple machines across separate platforms, a unified Python toolchain is desired that handles syncing of Python virtual environments, dependencies, and prescribes opinionated ways for running the project. We chose uv as the project orchestrator (handling project dependencies, defining and syncing Python environments), ty as the typechecker, and ruff as the linter and formatter. These run before the project can be run locally, but also before the documentation server can be built on pushing to the main branch.

On the software side of things we are behind the planned timetable by half a week, but there is hope that once deeper integration with control and electronics collaboration will speed up development. Remaining tasks include:

- Packaging the firmware and investigating a mechanism for the Pico to independently carry out sessions, decompressing the Programme into a target waveform and saving the signals from the run to disk. This should involve sensible warnings about disk space before starting the run. (May 25)
- Providing the control engineers with a main function with access to the pressure target over all time, previous actual pressure readings, and enable this function to send control signals to actuators (e.g. servos, stepper motors). In other words, wire up the firmware to accept an abstract control implementation. (May 25)
- Developing continuous integration/continuous deployment pipelines to compile the source code into executable binaries for all platforms (Windows, macOS, Linux), and present them as release artefacts

for consumption from the GitHub repository. (May 26)

- Wrapping the functions of the GUI in a CLI, such that: the composition of `Programme` files can be done via the command line; the starting of sessions can be done from the command line; and instantaneous readouts of signals can be done on the command line. (May 26)
- Investigate other blood flow simulators that may serve as sources to expand functionality of the `Programme` editor. (May 27)